

Lab Assignment 2

Christos Strydis
Rajendra Bishnoi
Amirreza Movahedin

2026

Contents

1	Introduction	2
2	Multiprocessing	2
3	Multithreading	4

Submission guidelines:

- Hand in the lab report in pdf format with the following name:
Lab2_Firstname_Lastname_Studentnumber (for example: Lab2_Pieter_deBoer_123456)
- Hand in the Python scripts (the **.py** files) as separate, unzipped Python scripts. Do not forget to add meaningful comments to your code. Handing in poorly annotated code will be penalized with one point.
- You must submit your code for exercises **2.6**, **2.7**, and **2.9** alongside your report for this lab. Please name your script files with the following convention: **L2.Qx** where x is the exercise number.

1 Introduction

The goal of this second lab assignment is to teach you safe parallel-programming techniques in Python for efficient execution of simulations. We will continue with the acceleration of The Virtual Brain (TVB) simulation, applying new parallelization techniques to enhance code execution time. As the previous lab, we will explore different aspects of the algorithm in the context of parallel programming. To refresh your memory on TVB and its algorithm, you can refer to the previous lab assignment.

2 Multiprocessing

Before we can define multiprocessing, we need to know what a process is. Technically speaking, a process is an instance of a computer program (a source code) being executed. Each process has a separate CPU core (worker) dedicated to it and its own memory space (i.e., separate piece of memory) reserved for storing the instructions and data needed for the process to run. This means that, when we perform *multiprocessing*, we distribute multiple tasks to several physical cores in our computer to execute them in parallel.

In the sequential version of the TVB algorithm, we used a single core of the computer to perform calculations. Like many other applications, the TVB algorithm consists of independent tasks that multiple CPU cores can work on. To map these tasks to said cores, we can use the most modern and user-friendly `concurrent.futures` module in Python. Avoid using the `multiprocessing` module, which is more cumbersome (but also more modular) to use.

When (re)structuring a piece of code to take advantage of multiprocessing, we implement the following execution steps:

- i. Initialize worker pool.
- ii. Submit tasks (to be executed asynchronously).
- iii. Wait for all of them to be completed.
- iv. When all tasks are completed, shut worker pool down.

In `concurrent.futures`, we start by creating a *pool of worker processes* which will carry out tasks submitted to it. We can explicitly create a `ProcessPoolExecutor` (an object that manages the processes), give it the tasks to be performed, and eventually shut it down. Thanks to the simple interface and the optimized back-end of this module, performing multiple tasks in parallel has now become very easy in Python. Besides convenience and uniformity of treatment (NB: there are other, more complex ways to handle processes individually), the process pool is also handy in maintaining a semi-permanent pool of available processes, as compared to constantly initializing and killing processes on the arrival of new tasks to execute. The reason is that every process uses resources (e.g., memory), and creating and killing these processes over and over takes time (overhead). Furthermore, the pool construct also predefines a maximum number of processes to be active at any point in time, placing an upper cap to the idling processes. Such process management is important since as the number of processes increases so does the difficulty of managing them.

A process pool is a convenient means of replicating the execution of a function (with different input data) across multiple processes (i.e., exploiting the underlying task-level, aka function-level, parallelism). The following example demonstrates the common practice of defining such functions in a module so that child processes can successfully import that module.

Exercise 2.1 (5 points)

Remember the TVB algorithm and its sequential implementation from the previous lab assignment, what challenges can you think of about parallelizing the code with multiprocessing? (**hint:** think about data dependencies that might exist within the algorithm.)

Exercise 2.2 (25 points)

Open the `tvb_par.py` script. This script implements a simple multiprocessing version of the TVB algorithm with MLP. Understand and compare this code with the sequential version of the algorithm (previous lab assignment).

1. What would happen if the following 2 lines of the code in the `simulate` function were swapped? (5 points)

```
with ProcessPoolExecutor() as executor:
    for t in range(1, total_timesteps):
```

2. Run the script for TVB76 and TVB192 dataset for 15ms. How does the execution times compare to the sequential version of TVB with MLP (The `tvb_seq_mlp.py` script)? How do you explain the difference? (5 points)
3. Modify the code and sweep the chunk size value between 2 and N . Plot the execution time for each of these datasets versus the chunk size. What are, approximately, the best chunk sizes for TVB76 and TVB192 simulations? Why does the performance of the code improve with larger chunk sizes? (5 points)
4. At the optimal chunk size, increase the simulation time from 15ms to 60ms. Does the execution time scales linearly? If not, how do you explain it? (5 points)
5. At the optimal chunk size, profile the code using `py-spy` and include a screenshot of the top function. For installing the library run:

```
pip install py-spy --break-system-packages
```

Describe what you observe in this profiling. (5 points)

Exercise 2.3 (7.5 points)

One of the main reasons behind the unimpressive performance of the multiprocessed code so far is the communication overhead. For each process, data should be copied to their memory. The `Xs` array specially takes a large amount of memory, and to copy it at each timestep introduces a large overhead. To mitigate this problem, one way is to use *shared memories*. With shared memories, we can only pass a reference to a block of memory to each worker and not copy the entire block. Additionally, the workers can modify the content of the shared memory and all other workers will be able to see the updated version. However, it is important to be aware of the hazards that come with multiple workers being able to alter the same addresses of memory, and if needed, locks must be used to ensure the integrity of the operations.

Open the `tvb_par_sm.py` script and examine it. For the shared memory, we are using the `shared_memory` module from the `multiprocessing` package. The memory blocks are now represented by NumPy arrays as they are better supported by the `shared_memory` module. Run this code for TVB76, TVB192, and TVB998 datasets for 15ms with the optimum chunk size you found in previous exercise (test different chunk sizes for TVB998 based on your observation for the other two datasets). How do the execution times compare to the sequential and multiprocessed implementation without shared memory?

Exercise 2.4 (7.5 points)

Although the use of shared memory improves the performance of the multiprocessed code, the `tvb_par_sm.py` script still has room for improvement. In the `center_task` function, each process at each timestep creates the attachment to the shared memory from scratch. This creates a large overhead which adds up when executed every timestep for each process. To address this, it is possible to run a piece of code at the beginning of each *worker*, instead of each *process*. This way, we can do the shared memory attachment only one time at the beginning of each worker's creation.

The `ProcessPoolExecutor` accepts an `initializer` function which is the piece of code each worker runs upon its creation. Open the `tvb_par_sm_imp.py` script and examine how this works.

Note the `global` identifier for the shared memories, which makes them visible for all workers and keeps the shared memory alive throughout the execution of the simulation, without the garbage collector reclaiming it.

Run this code for TVB76, TVB192, and TVB998 datasets for 15ms with the optimum chunk size you found in previous exercises. Compare the execution time of the simulation with the sequential implementation and the previous implementation with shared memory. How does it scale in comparison when increasing the number of centers?

Exercise 2.5 (5 points)

Based on what you have seen so far, in what cases do you think the multiprocessing parallelization technique would provide an advantage?

Exercise 2.6 (15 points)

Let's combine the multiprocessing technique we learned so far with the parallelization and acceleration techniques of the previous lab assignment. Implement a version of the algorithm where the workload of each center at each timestep is assigned to a single process (multiprocessing, what we have seen so far) and the calculations within the processes are vectorized. Use low-overhead shared memories and the sparse computation technique for the coupling calculation in this implementation.

Exercise 2.7 (15 points)

As you might remember from the brief introduction of The Virtual Brain, the reason we are interested in accelerating this application is the large number of simulations (in the range of thousands) needed per patient to create a personalized TVB model. That being said, another strategy that we can use in exploiting the multiprocessing capabilities of the system is to assign *the entirety of a simulation* to a process, and running multiple simulations in parallel using multiprocessing. In other words, we can increase the *throughput* (number of simulation timesteps per second) instead of reducing the *latency* of each simulation, which is what we have been doing so far.

Implement the mentioned version of TVB, where multiple simulations are launched in parallel using multiprocessing techniques you have seen so far. Use vectorization and sparse calculation techniques within each simulation. Do you think using shared memory is beneficial here to improve the performance of the code?

Report the execution time of the code for launching 200 simulations in parallel, each running the TVB192 dataset for 15ms. Use the same set of weight and distance matrices for all the simulations (in reality different set of parameters are used for different simulations). Compare the performance of this implementation with the version you implemented in Exercise 2.6 if it was to be executed 200 times back-to-back. Report the number of simulation iterations (timesteps) per second for each of the implementations. Which one performs better in terms of throughput? Why?

3 Multithreading

Each process has a corresponding thread, which is an independent path of execution within a program. It is possible for multiple threads to execute portions of one program and run concurrently. All threads that are part of a single program run in the same address space and are associated with the same process. In practice, this means that the code to be executed as well as all the variables declared in the program would be shared by all threads. We know that threads share the same memory space (i.e. they have access to the same variables), so special precautions must be taken so that two threads do not write to the same memory location at the same time; this can put the computer in an *indeterminate state*. The Global Interpreter Lock (GIL) is a mechanism of the Python Interpreter that takes care of this.

Practically speaking, multithreading in Python works very similarly to multiprocessing, from a user's perspective. For the same reasons described above for creating a pool of worker processes, here we create a pool of worker threads instead of creating single threads manually.

Exercise 2.8 (10 points)

Change the `tvb_par.py` script to run multithreading instead of multiprocessing. Run the multithreaded version for TVB76 and TVB192 datasets for 15ms.

1. Compare the execution time of the multithreaded version with the multiprocessed one. How do you explain the comparison results? (4 points)
2. Increase the simulation time from 15ms to 60ms. How does the execution time scale? Why does it scale differently than the multiprocessed implementation? (4 points)
3. Does the shared memory optimization we performed on the multiprocessed implementation also apply to the multithreaded version? Explain. (3 points)

Exercise 2.9 (10 points)

1. What is Python's Global Interpreter Lock (GIL)? (2.5 points)
2. Compare the execution time results from Exercise 2.8 with the results from the sequential implementation. How do you explain the results? (2.5 points)
3. Considering GIL, is it possible to improve the sequential version of TVB just by using multithreading? (2.5 points)
4. Are there any group of algorithms or applications that solely making them multithreaded in Python result in a faster version compared to their sequential implementation? (2.5 points)